



Prog II:

Herança, Polimorfismo e Funções Virtuais

Prof^a. Tainá Isabela

Introdução à Programação Orientada a Objetos

Juntamente com o conceito de classes e objetos, herança é, provavelmente, o aspecto mais importante em programação orientada a objetos (POO).

Herança permite **criar novas classes, chamadas de classes derivadas ou subclasses**, a partir das classes existentes, conhecidas como classes base.

A classe derivada herda todas as propriedades da classe base e, portanto, **especializa a classe base**, já que ela adiciona novas características ou faz refinamentos sobre a classe base. Observe que, nesse processo, a classe base **permanece inalterada**.

O Conceito de Herança

A herança é o mecanismo sintático e conceitual que permite a criação de novas classes a partir de classes já existentes, estabelecendo uma relação semântica do tipo "é um". A classe original é denominada **classe base, superclasse ou classe progenitora**, enquanto a nova estrutura é chamada de **classe derivada, subclasse ou classe filha**.

Através deste relacionamento, a classe derivada adquire **automaticamente os membros de dados e as funções-membro da classe base**, desde que os modificadores de acesso permitam. Esse processo **evita a duplicação de código** e estabelece uma hierarquia taxonômica clara dentro do domínio do problema computacional.

Importância da Herança

Herança é uma característica importante em POO, pois oferece suporte para **reusabilidade**. É importante observar que o reuso se dá não apenas no código já escrito e testado, mas também no componente implementado (que não requer nova implementação e, portanto, pode ser **reutilizado**).

Importante frisar ainda que uma classe já implementada pode também ser **adaptada para ser usada em situações diferentes**. Reusar o código existente economiza tempo e dinheiro. Adicionalmente, herança pode auxiliar na modelagem e solução de um problema.

```
#include <iostream>
using namespace std;

// Classe base
class Pessoa {
public:
    void apresentar() {
        cout << "Eu sou uma pessoa." << endl;
    }
protected:
    int idade;
private:
    string nome;
};

// Classe derivada
class Professor : public Pessoa {
public:
    void ensinar() {
        cout << "Eu sou um professor." << endl;
    }
};
```

```
#include <iostream>
#include <vector>
using namespace std;

class Pessoa {
public:
    virtual void apresentar() const {
        cout << "Eu sou uma pessoa." << endl;
    }
    virtual ~Pessoa() = default; // Destrutor virtual é boa prática
};

class Professor : public Pessoa {
public:
    void apresentar() const override {
        cout << "Eu sou um professor." << endl;
    }
};

class Aluno : public Pessoa {
public:
    void apresentar() const override {
        cout << "Eu sou um aluno." << endl;
    }
};
```

```
int main() {  
    vector<Pessoa*> pessoas;  
    pessoas.push_back(new Pessoa());  
    pessoas.push_back(new Professor());  
    pessoas.push_back(new Aluno());  
  
    for (const Pessoa* p : pessoas) {  
        p->apresentar(); // Chama o método correto conforme o tipo real do objeto  
    }  
  
    // Liberta memória  
    for (Pessoa* p : pessoas) delete p;  
    return 0;  
}
```

Modificadores de Acesso e a Visibilidade na Herança

A visibilidade dos membros herdados é governada de maneira estrita pelos qualificadores de acesso, sendo eles **público (public)**, **protegido (protected)** e **privado (private)**. Membros **públicos permanecem acessíveis por qualquer função externa**, enquanto membros **privados são estritamente confinados à classe que os declarou**, tornando-se inacessíveis para as subclasses.

O modificador protegido atua de forma intermediária, garantindo o **encapsulamento contra o escopo externo, mas permitindo o acesso direto pelas classes derivadas**. A escolha correta do tipo de herança altera o nível de visibilidade dos membros na classe filha, impactando diretamente o design do acoplamento e a segurança do código.

Tabela 9.1

Especificador de acesso	Acessível da própria classe	Acessível da classe derivada	Acessível a objetos de fora da classe
public	sim	sim	sim
protected	sim	sim	não
private	sim	não	não

O Princípio da Substituição de Liskov

Formulado por Barbara Liskov, este princípio constitui a base teórica para a correta utilização da herança em sistemas orientados a objetos. O axioma estipula que se uma classe **S** é uma subclasse de **T**, então os objetos do tipo **T** podem ser substituídos por objetos do tipo **S** sem alterar as propriedades desejáveis do programa.

Em termos práticos, uma classe derivada **não deve violar as expectativas de comportamento criadas pela classe base**, nem restringir suas precondições. A violação deste princípio gera abstrações incoerentes, onde subtipos quebram a execução do sistema quando submetidos a **polimorfismo**.

Conceito e Tipos de Polimorfismo

O polimorfismo, termo originário do grego que significa "muitas formas", refere-se à capacidade de um mesmo identificador ou interface associar-se a **diferentes comportamentos dependendo do contexto**. Na ciência da computação, classifica-se o polimorfismo em **estático** (ou em tempo de compilação) e **dinâmico** (ou em tempo de execução).

O **polimorfismo estático** manifesta-se através da **sobrecarga de funções e do uso de modelos/templates**, onde o compilador resolve as chamadas com base nos tipos dos argumentos. Já o **polimorfismo dinâmico** depende fundamentalmente da **herança e da sobrescrita de métodos**, permitindo que a decisão sobre qual função executar ocorra estritamente durante a execução do programa.

Mecanismo do Polimorfismo Dinâmico

O polimorfismo dinâmico viabiliza-se quando ponteiros ou referências para uma classe base apontam para instâncias de classes derivadas em tempo de execução. O compilador, ao encontrar uma chamada de método através desse ponteiro genérico, suspende a vinculação estática tradicional da linguagem.

Essa técnica concede flexibilidade ao sistema, permitindo que coleções **heterogêneas** de objetos sejam processadas uniformemente por algoritmos genéricos. Por exemplo, uma lista de ponteiros do tipo genérico **Forma** pode conter instâncias de **Circulo** e **Quadrado**, sendo processada por um **laço único** que dispara **comportamentos específicos** de cada figura geométrica.

O Papel das Funções Virtuais

Para que o polimorfismo dinâmico funcione como esperado, é obrigatória a declaração de funções virtuais na classe base utilizando a palavra-chave **virtual**. Uma função virtual sinaliza ao compilador que a vinculação daquela chamada deve ser adiada até o **tempo de execução** (late binding).

Quando uma classe derivada sobrescreve essa função, o comportamento correspondente à instância real do objeto é invocado, mesmo que a **chamada ocorra por meio de uma referência da superclasse**. Sem a diretiva virtual, ocorre o fenômeno de **ocultação de método** (shadowing), onde a semântica da classe base prevalece estaticamente.

Conceito de Classes Abstratas

Uma **classe abstrata** é uma **estrutura conceitual** projetada exclusivamente para servir como **modelo** para outras classes, sendo **expressamente proibida a sua instanciação direta** no sistema. Ela atua como uma interface de alto nível, definindo um contrato de **comportamento obrigatório** que todas as suas subclasses devem cumprir.

Na arquitetura de software, as classes abstratas são utilizadas para **capturar generalizações abstratas** do mundo real que não possuem existência concreta por si só. A tentativa de criar um objeto a partir de uma classe abstrata resulta em um erro de compilação, protegendo o sistema contra o uso inadequado de conceitos incompletos.

Funções Virtuais Puras

A abstração de uma classe é formalizada tecnicamente através da declaração de pelo menos uma função virtual pura. Em C++, uma função virtual pura é definida atribuindo-se a expressão **= 0 ao final de sua assinatura**, indicando que a classe base não fornece nenhuma implementação padrão para aquele método.

As subclasses que herdam dessa classe abstrata são obrigadas a sobrescrever e implementar todas as funções virtuais puras herdadas, **caso contrário, também se tornarão classes abstratas**. Se uma classe contiver exclusivamente funções virtuais puras e nenhum membro de dados, ela assume o papel de uma interface pura.

```
class Forma {  
public:  
    // Função virtual pura  
    virtual double area() = 0;  
    virtual double perimetro() = 0;  
};
```

```
class Circulo : public Forma {  
private:  
    double raio;  
public:  
    Circulo(double r) : raio(r) {}  
    double area() override {  
        return 3.1415 * raio * raio;  
    }  
    double perimetro() override {  
        return 2 * 3.1415 * raio;  
    }  
};
```



```
int main() {  
    Circulo c(5.0);  
    Forma* f = &c;  
    std::cout << "Área: " << f->area() << std::endl;  
    std::cout << "Perímetro: " << f->perimetro() << std::endl;  
    return 0;  
}
```

O Problema do Destrutor Não-Virtual

Um erro crítico de gerenciamento de memória ocorre quando uma classe possui funções virtuais, mas o seu destrutor é **declarado de forma convencional**, ou seja, **não-virtual**. Se um objeto da classe derivada for deletado através de um ponteiro da classe base, o compilador invocará estaticamente apenas o destrutor da classe base.

Como consequência direta, os recursos específicos alocados pela classe derivada (como memória dinâmica ou descritores de arquivos) não serão liberados, gerando vazamento de memória (memory leak). A regra de ouro do design orientado a objetos dita que **se uma classe possui qualquer função virtual, seu destrutor deve obrigatoriamente ser virtual**.

Arquitetura de Drivers de Dispositivos

Uma aplicação prática clássica do polimorfismo e das classes abstratas encontra-se no desenvolvimento de sistemas operacionais e drivers de dispositivos de hardware:

Define-se uma classe abstrata **DispositivoArmazenamento** com funções virtuais puras como **ler()** e **gravar()**. Fabricantes de hardware implementam subclasses específicas como **DiscoRigido** ou **UnidadeSSD**, fornecendo o código de baixo nível necessário para manipular seus dispositivos.

O núcleo do sistema operacional interage exclusivamente com a interface abstrata através de ponteiros genéricos, operando de maneira completamente agnóstica em relação ao hardware subjacente conectado ao computador.

Motores de Jogos e o Loop de Atualização

Os motores de jogos (game engines) utilizam extensivamente o polimorfismo dinâmico para gerenciar as entidades visuais e lógicas presentes no cenário virtual. Uma classe base abstrata **Actor** ou **GameObject** define funções virtuais puras como **update**(float deltaTime) e **render**().

O motor mantém um vetor dinâmico composto por ponteiros para a classe base **GameObject**, populado com instâncias **heterogêneas** de Jogador, Inimigo, Projétil e ElementoCenário. A cada iteração do ciclo principal do jogo, o motor percorre este vetor invocando as funções virtuais, **fazendo com que cada entidade atualize sua física e execute sua rasterização de forma autônoma.**

A Diretiva Override

A sobrescrita de métodos (method overriding) ocorre quando a classe derivada **reimplementa uma função virtual declarada na classe base**, mantendo estritamente **a mesma assinatura, tipo de retorno e qualificadores**.

Para evitar erros sutis de compilação, como pequenas diferenças na grafia do nome da função ou nos tipos dos parâmetros que **resultariam em sobrecarga e não em sobrescrita**, os padrões modernos introduziram o especificador **override**. Ao adicionar override ao final da declaração na subclasse, o programador instrui **o compilador a validar se aquela função realmente substitui um método virtual da classe base**, abortando a compilação em caso negativo.

```
class IAnimal {  
public:  
    virtual void falar() = 0; // Método puro: = 0  
    virtual ~IAnimal() = default; // Destrutor virtual  
};
```

```
class Gato : public IAnimal {  
public:  
    void falar() override {  
        std::cout << "Miau!" << std::endl;  
    }  
};
```

```
class Cachorro : public IAnimal {  
public:  
    void falar() override {  
        std::cout << "Au au!" << std::endl;  
    }  
};
```

```
int main() {  
    IAnimal* animal1 = new Gato();  
    IAnimal* animal2 = new Cachorro();  
  
    animal1->falar(); // Saída: Miau!  
    animal2->falar(); // Saída: Au au!  
  
    delete animal1;  
    delete animal2;  
    return 0;  
}
```

O Mecanismo de Herança Múltipla

A herança múltipla ocorre quando uma subclasse **herda diretamente as propriedades e comportamentos de mais de uma superclasse simultaneamente**. Embora confira grande poder de expressão, a herança múltipla introduz uma ambiguidade clássica na computação gráfica conhecida como o **"Problema do Diamante"**.

Esse problema ocorre quando **duas classes derivadas herdam de uma mesma classe base comum** e, por sua vez, **uma quarta classe herda das duas classes intermediárias**. Se um método da classe base original for invocado pela quarta classe, o compilador enfrentará uma ambiguidade sobre qual caminho de herança seguir, exigindo o uso de herança virtual para unificar as instâncias duplicadas.

Vantagens das Abstrações Polimórficas

A principal vantagem da utilização integrada de herança, polimorfismo e funções virtuais reside no cumprimento do Princípio do Aberto/Fechado (Open/Closed Principle). **Os sistemas tornam-se abertos para extensão**, mas fechados para modificação, o que significa que novas funcionalidades e novos tipos de objetos podem ser adicionados ao ecossistema **sem a necessidade de alterar ou recompilar o código estrutural já testado**.

Isso aumenta a manutenibilidade, promove o reaproveitamento de componentes lógicos e reduz a incidência de regressões de bugs em sistemas de larga escala.

Questionário



Exercícios

1. Criar Classe Base e Derivada Implemente uma classe base chamada Veiculo contendo um atributo público do tipo texto marca. Em seguida, crie uma classe derivada chamada Carro que herde publicamente de Veiculo e adicione um atributo inteiro específico chamado numeroPortas.
2. Implementar Função Virtual Declare na classe base Veiculo uma função membro chamada emitirSom(), configurando-a como uma função virtual que exiba na tela uma mensagem genérica sobre o ruído do motor.
3. Sobrescrever Método Na classe derivada Carro, realize a sobrescrita (override) do método virtual emitirSom(). A nova implementação deve exibir na tela uma mensagem específica simulando o som de uma buzina automotiva.